

République du Cameroun

Republic du Cameroon

Paix – Travail – Patrie

Peace – Work – Fatherland

Ministère de l'Education Secondaire

Ministry of Secondary Education

Institut Universitaire Saint Jean

Saint Jean Institute



Thème :

## CAHIER DE CHARGE : MYPING

*Supervisé par:*

**Mr. NGUIMBUS Emmanuel**

*Fait par :*

- ESSAMA AXEL MERLIN
- MENGA MARCO ROMUALD
- MOUGANG NGUETI YAN KEMUEL
- TAGNE JOSEPH NATHANAEL
- TCHATCHOUA NGASSAM GIOVANNI NESTOR

# SOMMAIRE

|                                               |     |
|-----------------------------------------------|-----|
| <u>LISTE DES TABLEAUX</u> .....               | ii  |
| <u>ABREVIATIONS</u> .....                     | iii |
| <u>GLOSSAIRE</u> .....                        | iv  |
| <u>1. INTRODUCTION GENERALE</u> .....         | 1   |
| <u>2. ANALYSE DE L’EXISTANT</u> .....         | 2   |
| <u>3. SPECIFICATION</u> .....                 | 4   |
| <u>4. CONTRAINTES</u> .....                   | 8   |
| <u>5. PLAN DE TESTS ET VALIDATION</u> .....   | 9   |
| <u>6. ORGANISATION ET PLANIFICATION</u> ..... | 11  |
| <u>7. LIVRABLES ATTENDUS</u> .....            | 12  |

# LISTE DES TABLEAUX

|                                               |    |
|-----------------------------------------------|----|
| Tableau 1 : Champ ICMP.....                   | 3  |
| Tableau 2 : Options du programme Myping ..... | 4  |
| Tableau 3 : Contraintes techniques .....      | 8  |
| Tableau 4 : Tests fonctionnelles .....        | 10 |
| Tableau 5 : Management de l'équipe.....       | 11 |
| Tableau 6 : Planning Prévisionnelle.....      | 11 |

# ABBREVIATIONS

- ❖ **CLI** : Commande Line Interface
- ❖ **DNS** : Domain Name System
- ❖ **ICMP** : Internet Control Message Protocol
- ❖ **Mdev** : Mean Deviation
- ❖ **RTT** : Round Trip Time
- ❖ **TTL** : Time To Live

# GLOSSAIRE

- ❖ **CAP\_NET\_RAW** : Capacité linux permettant d'ouvrir des raw sockets sans être root
- ❖ **Checksum** : Somme de contrôle calculé sur les données d'un paquet pour détecter les erreurs de transmission
- ❖ **CLI** : Interface en ligne de commande
- ❖ **DNS** : Système de résolution de noms de domaine en adresses IP
- ❖ **ICMP** : Protocole de couche réseau pour les messages de contrôle de d'erreur
- ❖ **mdev** : écart moyen des RTT par rapport à la moyenne ; indicateur de stabilité réseau
- ❖ **Raw Socket** : Socket permettant l'accès direct aux couches réseau sans encapsulation protocolaire supplémentaire
- ❖ **RTT** : Temps de trajet aller-retour d'un paquet réseau, exprimé en millisecondes
- ❖ **SIGINT** : Signal d'interruption UNIX envoyé par la combinaison de touche « CTRL+C » pour demander l'arrêt d'un processus
- ❖ **TTL** : Compteur décrémenté à chaque saut réseau ; le paquet est rejeté lorsqu'il atteint 0

# 1. INTRODUCTION GENERALE

## 1.1. Contexte Générale

Le projet **MYPING** a été réalisé dans le cadre du cours de « Théorie des Systèmes d'Exploitation », et a pour but de reproduire la commande « ping » en utilisant le langage C.

La commande **ping** est l'un des outils les plus utilisés par les administrateurs réseau et les développeurs, pour vérifier la connectivité réseau entre différents services ou postes de travail. Son fonctionnement repose sur le protocole **ICMP(Internet Control Message Protocol)**. Cette commande envoie des paquets dits **Echo Request** vers une machine et attend en retour des **Echo Reply**.

## 1.2. Objectif du projet

L'objectif central de ce projet est : **Reproduire le comportement de la commande ping tout en conservant toutes ses fonctionnalités.**

En d'autres termes, nous cherchons à :

- Comprendre le plus possible le fonctionnement du protocole ICMP
- Reproduire l'interface en ligne de commande de la commande conformément au standard Unix
- Calculer et afficher les statistiques réseau (RTT, taux de gain/pertes)
- Maîtriser la manipulation des sockets sous linux

## 2. ANALYSE DE L'EXISTANT

### 2.1. Histoire de la commande Ping

La commande **ping** a été écrite par **Mike Muuss en 1983**, alors qu'il travaillait pour le Ballistic Research Laboratory. L'histoire raconte qu'il s'est inspiré de la métaphore du sonar pour imaginer un outil réseau capable de "détecter" la présence d'une machine distante. Le nom **Packet Internet Groper** lui fut attribué après coup, par rétro-acronymie.

Aujourd'hui, ping est présent dans tous les systèmes d'exploitation modernes : Linux, macOS, Windows. Si le comportement de base est identique partout, les implémentations diffèrent légèrement selon les plateformes, notamment dans le format d'affichage et les options disponibles. C'est l'une des raisons pour lesquelles nous avons choisi de cibler exclusivement Linux pour ce projet.

### 2.2. Protocol ICMP

ICMP est défini par la **RFC 792**. Il opère à la couche réseau (couche 3 du modèle OSI) et est encapsulé directement dans les datagrammes IP. Pour le diagnostic réseau, deux types de messages nous intéressent particulièrement :

- Type 8 (**Echo Request**) : c'est le paquet que notre programme envoie
- Type 0 (**Echo Reply**) : c'est la réponse que la machine cible nous renvoie
- Type 11 (**Time Exceeded**) : c'est la réponse reçue lorsque le TTL tombe à 0

Chaque paquet ICMP Echo contient les champs suivants :

| <b>Champ</b>    | <b>Taille</b> | <b>Description</b>                                     |
|-----------------|---------------|--------------------------------------------------------|
| Type            | 8 bits        | Type du message (8==Request, 0==Reply)                 |
| Code            | 8 bits        | Sous-type, toujours 0 pour Echo                        |
| Checksum        | 16 bits       | Somme de contrôle d'intégrité du paquet                |
| Identifiant     | 16 bits       | Identifiant de session                                 |
| Sequence Number | 16 bits       | Numéro de séquence croissant pour chaque paquet envoyé |
| Timestamp       | 64-128 bits   | Heure d'envoi pour calculer le RTT                     |
| Data            | Variable      | Données de charge utile                                |

*Tableau 1 : Champ ICMP*

**NB :** Une erreur dans le calcul du checksum ICMP rend le paquet invalide et la machine distante l'ignorera silencieusement ce qui peut mener à des heures de débogage si l'on n'y prend pas garde.



# 3. SPECIFICATION

## 3.1. Spécification Fonctionnelles

### 3.1.1. Fonctionnalités Obligatoires

#### 3.1.1.1. *Envoie de paquets ICMP Echo Request*

Cette fonctionnalité est le cœur même de notre projet. Myping doit être capable de construire entièrement un paquet ICMP(en-tête compris) et de l'envoyer via un socket a une adresse IP ou un nom d'hôte. Avant tout envoie, le programme devra résoudre le nom d'hôte en adresse IP via DNS puis afficher cette adresse à l'utilisateur.

#### 3.1.1.2. *Réception et traitement des réponses*

Une fois les paquets envoyés, il faut les traiter correctement a la réception. Le programme doit écouter les réponse ICMP et les associer aux requêtes correspondantes grâce à l'identifiant de session et au numéro de séquence.

#### 3.1.1.3. *Option de la ligne de commande*

Notre programme « MYPING » devra supporter les options suivantes :

| OPTION | PARAMETRE | DESCRIPTION                                                   |
|--------|-----------|---------------------------------------------------------------|
| -c     | count     | Nombre de paquet à envoyer                                    |
| -i     | Interval  | Intervalle en secondes entre deux envoies                     |
| -t     | TTL       | Valeur du TTL dans les paquets IP                             |
| -s     | Size      | Taille du payload en octets                                   |
| -W     | Timeout   | Délai d'attente maximal pour une réponse                      |
| -q     | -----     | Mode silencieux ; affiche uniquement les statistiques finales |
| -v     | -----     | Mode verbeux ; informations supplémentaires pour le débogage  |

Tableau 2 : Options du programme Myping

#### 3.1.1.4. *Affichage des statistiques*

A la fin de l'exécution, que ce soit à la suite de la combinaison de touche « CTRL+C » ou après l'envoi du nombre de paquets demander, le programme affichera un récapitulatif de session. Le format visé est celui de la commande « ping ».

**Format par paquet reçu :**

```
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=12.4 ms
```

**Format du bloc de statistiques finales :**

```
--- 8.8.8.8 myping statistics ---
```

```
5 packets transmitted, 5 received, 0% packet loss, time 4012ms
```

```
Rtt min/avg/max/mdev = 10.2/12.1/15.8/1.9 ms
```

#### 3.1.2. **Fonctionnalités Secondaires**

Dans la mesure d'une future amélioration future, nous nous sommes penchez sur les points suivants :

- Ajout de l'option -f (flood ping) pour les tests de charge réseau
- Résolution inverse d'adresse IP pour afficher le nom d'hôte source
- Export des résultats dans un fichier CSV bien structurer ou JSON pour faciliter l'analyse post-test

## 3.2. Spécification non fonctionnelles

### 3.2.1. Performance

La précision de mesure du RTT est un critère clé car ici, on vise une erreur de mesure inférieure à 1 milliseconde dans des conditions réseau normales. Pour cela, on utilisera le « gettimeofday() » qui offre une résolution en nanosecondes et n'est pas sensible aux ajustements de l'horloge système.

Le RTT est calculé en comparant l'heure d'envoi(stocké dans le paquet) avec l'heure de réception(capturé via gettimeofday() ). La formule est la suivante :

$$\text{RTT (ms)} = (\text{fin.tv\_sec} - \text{debut.tv\_sec}) \times 1000 + (\text{fin.tv\_usec} - \text{debut.tv\_usec}) / 1000$$

En générale, ce qui est visé sont les points suivants :

- ✓ Précision RTT : erreur < 1ms
- ✓ Pas de fuite mémoire sur des exécutions longues
- ✓ Consommation CPU minimale en mode attente entre les envois

### 3.2.2. Fiabilité et Robustesse

Un programme réseau doit savoir gérer l'inattendu. Par conséquent, nous prêtons une attention particulière aux points suivants :

- Gestion propre du signal SIGINT(Ctrl+C)
- affichage des statistiques
- Utilisation systématique des timeouts
- Validation rigoureuse de tous les arguments de la ligne de commande
- Gestion explicite des erreurs réseau (hôte et réseau inaccessible, TTL expiré)
- Vérification obligatoire du checksum ICMP à la réception

### 3.2.3. Portabilité

Notre programme « myping » visera les systèmes Linux avec un noyau 4.x et plus. Nous n'utiliserons aucunes bibliothèques tierce non justifiée et si une dépendance externe s'avère nécessaire, elle sera documentée et son ajout fera l'objet d'une discussion collective.

### 3.2.4. Sécurité

L'utilisations des raw sockets implique l'utilisation des privilèges élevés. Nous adopterons le principe de moindre privilèges cette a dire que les droits root ne seront demandé que le temps d'ouvrir le socket, puis abandonnés via setuid/setgid. Le programme ne devra jamais modifier de fichiers systèmes sans accord explicite de l'utilisateur.

### 3.2.5. Maintenabilité

Pour faciliter la collaboration et la reprise du code par d'autres personnes :

- Le code sera commenté, structuré en module cohérents
- Un fichier « Makefile » sera fournie pour faciliter la compilation du programme
- Un guide documentera la compilation, l'installation et l'utilisation

## 4. CONTRAINTES

### 4.1. Contraintes Techniques

| Contexte               | Contrainte                      | Justification                                                                      |
|------------------------|---------------------------------|------------------------------------------------------------------------------------|
| Système d'exploitation | Linux (Kali linux recommander)  | Les sockets RAW ICMP ne sont pas directement compatibles avec les systèmes Windows |
| Droit système          | Exécution avec sudo obligatoire | Les sockets RAW nécessitent les privilèges root                                    |
| Protocole réseau       | IPv4 uniquement                 | IPv6 (ICMP version 6) est hors du périmètre de ce projet                           |
| Compilateur            | GCC avec les flags -Wall -O2    | Assure la détection des erreurs et l'optimisation du binaire                       |

*Tableau 3 : Contraintes techniques*

### 4.2. Contraintes de sécurité

L'utilisation de sockets RAW impose plusieurs précautions de sécurité importantes dans le contexte réseau :

- **Privilège minimum** : le programme doit uniquement demander les droits root pour créer le socket, puis les utiliser de façon responsable.
- **Filtrage des paquets** : le socket RAW reçoit tous les paquets ICMP destiné à la machine et doit les filtrer par identifiant (PID) pour ne traiter que ses réponse.
- **Validation du checksum** : tous les paquets dont le checksum est invalide est considéré comme corrompu et est ensuite ignoré.

# 5. PLAN DE TESTS ET VALIDATION

## 5.1. Test fonctionnelles

Chaque fonctionnalité fera l'objet d'un cas de test documenté avant même d'être développé :

| ID   | Tache                        | Commande                                        | Résultat attendu                                                                     |
|------|------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------|
| T-01 | Ping réseau local            | <code>sudo ./myping 192.168.0.5</code>          | Réponse avec $RTT < 5ms$ , 0% perte                                                  |
| T-02 | Ping Internet                | <code>sudo ./myping google.com</code>           | Résolution DNS + réponse avec $RTT < 100ms$                                          |
| T-03 | Ping avec l'option -c        | <code>sudo ./myping -c 4 192.168.10.6</code>    | Exactement 4 paquets envoyés puis arrêt                                              |
| T-04 | Ping avec l'option -i        | <code>sudo ./myping -c 3 -i 2 google.com</code> | 2 secondes d'intervalle lors de l'envoi de chaque paquet                             |
| T-05 | Ping avec l'option -s        | <code>sudo ./myping -s 128 facebook.com</code>  | Paquets de 136 octets (128+8) affiché                                                |
| T-06 | Ping avec l'option -t        | <code>sudo ./myping -t 1 youtube.com</code>     | Time Exceeded ou aucune réponse, si le $TTL=1$                                       |
| T-07 | Ping avec l'option -w        | <code>sudo ./myping -w 3 172.10.15.0</code>     | Arrêt après 3 secondes d'attente maximum                                             |
| T-08 | Ping avec l'option -q        | <code>sudo ./myping -q -c 3 10.10.1.1</code>    | Aucun affichage pendant le ping, seulement le résumer à la fin                       |
| T-09 | Ping avec l'option -n        | <code>sudo ./myping -n 8.8.8.8</code>           | Affichage IP brute sans résolution DNS                                               |
| T-10 | Combinaison de touche Ctrl+C | <code>sudo ./myping 10.0.0.5</code> puis Ctrl+C | Affichage du résumé et programme terminé lorsque la combinaison de touche est entrée |
| T-11 | Ping vers un hôte inexistant | <code>sudo ./myping [hôte inexistant]</code>    | Message d'erreur DNS, retour code 1                                                  |

|      |                               |                               |                                                     |
|------|-------------------------------|-------------------------------|-----------------------------------------------------|
| T-12 | Ping vers un hôte injoignable | sudo ./myping 192.168.100.254 | Timeout à chaque paquet (100% perte, retour code 1) |
|------|-------------------------------|-------------------------------|-----------------------------------------------------|

*Tableau 4 : Tests fonctionnelles*

## 5.2. Critères de validation

- Compilation : aucune erreur avec gcc -Wall -O2
- Fonctionnel : tous les tests (de T-01 à T-12) passent
- Statistiques : les valeurs RTT sont cohérentes
- Signal : Ctrl+C affiche bien les statistiques avant de quitter

## 5.3. Comparaison avec ping natif

L'un de nos critères de validation les plus importantes sera **la conformité avec la commande ping native**. Nous lancerons les deux outils sur les mêmes hôtes et comparerons les RTT mesuré et les statistiques de perte de paquets. Les valeurs obtenues devront être dans la même plage compte tenu de la variable naturelle de réseau.

# 6. ORGANISATION ET PLANIFICATION

## 6.1. Répartition des taches

L'équipe 10 est composé de plusieurs membres. Voici les rôles que nous avons défini pour assurer une progression parallèle du projet :

| Rôle                  | Responsabilités                                                           |
|-----------------------|---------------------------------------------------------------------------|
| Chef de projet        | Coordination générale, rédaction du cahier des charges, suivi du planning |
| Développeur réseau    | Raw socket, envoie/réception ICMP, résolution DNS                         |
| Développeur protocole | Construction et décodage des paquets ICMP, calcul du checksum             |
| Développeur CLI       | Modules statistiques, affichage                                           |
| Responsable tests     | Ecriture et exécution du plan de tests et validation                      |

Tableau 5 : Management de l'équipe

## 6.2. Planning Prévisionnel

| Phase   | Activités                                                   |
|---------|-------------------------------------------------------------|
| Phase 1 | Analyse, spécification, choix technologiques                |
| Phase 2 | Mise en place de l'environnement, structuration du projet   |
| Phase 3 | Implémentation core : socket, ICMP, envoi/réception de base |
| Phase 4 | Implémentations des options CLI et module statistiques      |
| Phase 5 | Tests, débogage, comparaison avec ping native               |
| Phase 6 | Rédaction du rapport finale, préparation de la présentation |

Tableau 6 : Planning Prévisionnelle



## 6.3. Outils et environnement de développement

Nous avons choisi des outils éprouvés que la majorité de l'équipe connaît déjà :

- Systèmes d'exploitation : Kali Linux 23 ou supérieur
- Compilateur : GCC avec les flags -Wall
- Gestion de version : Git, avec dépôt partagé sur GitHub
- Outil de build : Make
- Communication d'équipe : Notion, WhatsApp

# 7. LIVRABLES ATTENDUS

## 7.1. Code Source

- ✓ Code source complet, compilable via la commande make
- ✓ Makefile avec les cibles : all, clean, fclean, test
- ✓ Respect des normes de codage définies collectivement

## 7.2. Documentation

- ✓ Procedure\_d'execution.txt : compilation, installation, exemples d'utilisation
- ✓ Commentaires de code sur toutes les fonctions et structures de données complexes
- ✓ Manuel d'utilisation en ligne accessible via l'option -h

# Table des matières

|                                             |     |
|---------------------------------------------|-----|
| SOMMAIRE .....                              | i   |
| LISTE DES TABLEAUX.....                     | ii  |
| ABREVIATIONS.....                           | iii |
| GLOSSAIRE .....                             | iv  |
| 1. INTRODUCTION GENERALE .....              | 1   |
| 1.1. Contexte Générale .....                | 1   |
| 1.2. Objectif du projet.....                | 1   |
| 2. ANALYSE DE L'EXISTANT .....              | 2   |
| 2.1. Histoire de la commande Ping .....     | 2   |
| 2.2. Protocol ICMP .....                    | 2   |
| 3. SPECIFICATION.....                       | 4   |
| 3.1. Spécification Fonctionelles .....      | 4   |
| 3.1.1. Fonctionnalités Obligatoires .....   | 4   |
| 3.1.2. Fonctionnalités Secondaires.....     | 5   |
| 3.2. Spécification non fonctionnelles ..... | 6   |
| 3.2.1. Performance .....                    | 6   |
| 3.2.2. Fiabilité et Robustesse .....        | 6   |
| 3.2.3. Portabilité .....                    | 7   |
| 3.2.4. Sécurité .....                       | 7   |
| 3.2.5. Maintenabilité .....                 | 7   |
| 4. CONTRAINTES .....                        | 8   |
| 4.1. Contraintes Techniques.....            | 8   |
| 4.2. Contraintes de securite .....          | 8   |
| 5. PLAN DE TESTS ET VALIDATION.....         | 9   |
| 5.1. Test fonctionnelles .....              | 9   |
| 5.2. Critères de validation .....           | 10  |
| 5.3. Comparaison avec ping natif .....      | 10  |

|      |                                               |    |
|------|-----------------------------------------------|----|
| 6.   | ORGANISATION ET PLANIFICATION.....            | 11 |
| 6.1. | Répartition des tâches.....                   | 11 |
| 6.2. | Planning Prévisionnel.....                    | 11 |
| 6.3. | Outils et environnement de développement..... | 12 |
| 7.   | LIVRABLES ATTENDUS .....                      | 12 |
| 7.1. | Code Source .....                             | 12 |
| 7.2. | Documentation.....                            | 12 |